

Módulo 7 – Capítulo 05 – Sección 01

LangGraph: grafos de estado para flujos agénticos deterministas y cíclicos

El capítulo anterior construyó los componentes internos del agente: razonamiento, herramientas, memoria. Este capítulo examina la infraestructura que organiza esos componentes en sistemas cohesivos y operables: los frameworks de agentes. LangGraph es el framework más adecuado cuando el flujo de control del agente es complejo —ciclos, paralelismo, subgrafos, interrupciones para revisión humana— y cuando la auditoría de cada transición de estado es un requisito operativo. Comprender LangGraph es comprender el enfoque de control máximo en la ingeniería agéntica moderna.

LangGraph es el framework de agentes de LangChain que modela el comportamiento agéntico como un grafo de estado explícito: cada nodo es una función Python que recibe el `AgentState` actual, lo procesa, y devuelve un nuevo estado; cada arista (directa o condicional) define las transiciones entre nodos. A diferencia del `AgentExecutor` de LangChain clásico —que usa un bucle `while` implícito difícil de inspeccionar y modificar—, LangGraph expone el grafo completo del agente como un objeto que puede ser visualizado, compilado, y ejecutado con control preciso sobre cada transición. Un grafo de LangGraph es directamente legible como arquitectura: se puede ver exactamente qué nodos existen, qué condiciones determinan cada transición, y cuáles son los estados terminales posibles.

El componente central es el `StateGraph` con su `AgentState`. El estado se define como un `TypedDict` con los campos que deben persistir entre nodos: historial de mensajes, variables de contexto de la tarea, resultados intermedios, contadores de iteración. Cada campo puede tener una función de reducción que define cómo se combina el nuevo valor con el valor anterior: por defecto, el nuevo valor reemplaza al anterior; para listas como el historial de mensajes, la reducción habitual es concatenar (añadir los nuevos mensajes al historial existente). Los nodos se registran con `graph.add_node(name, function)` y las aristas con `graph.add_edge(from_node, to_node)` para aristas directas o `graph.add_conditional_edges(from_node, condition_fn, {outcome: next_node})` para

aristas condicionales. La `condition_fn` inspecciona el estado actual y devuelve la clave del próximo nodo.

Las **aristas condicionales** son la característica que hace a LangGraph fundamentalmente diferente de un pipeline secuencial. En un agente ReAct implementado en LangGraph, la arista condicional desde el nodo de razonamiento determina si el agente debe invocar una herramienta (el LLM decidió hacer una tool call) o terminar (el LLM decidió dar la respuesta final al usuario). Esta condición no está hardcodeada en la lógica del agente; está representada como una función Python en el grafo que es auditable, testeable de forma independiente, y modificable sin tocar el resto del flujo. Esta separación entre el flujo de control y la lógica de cada nodo es la propiedad más valiosa de LangGraph para sistemas de producción que evolucionan.

El **checkpointing** es la función que transforma LangGraph de un framework de agentes a un sistema de agentes stateful en producción. Al compilar el grafo con `graph.compile(checkpointer=SqliteSaver.from_conn_string("agents.db"))` y ejecutar con un `thread_id` específico, LangGraph serializa el `AgentState` completo a la base de datos después de cada nodo. Esto permite: reanudar sesiones interrumpidas exactamente desde el último nodo completado, ejecutar múltiples sesiones concurrentes del mismo agente con estado independiente, y hacer debugging reproduciendo la ejecución desde cualquier estado guardado. En producción con `PostgresSaver`, el checkpointing provee la misma garantía con alta disponibilidad y sin el límite de escala de SQLite.

El **streaming de pasos** con `graph.stream(input, config)` devuelve un generador que emite el estado después de cada nodo, permitiendo que el cliente vea el progreso del agente en tiempo real. Esto es fundamentalmente diferente a esperar el resultado final: el usuario puede ver "Buscando información... Analizando resultados... Generando respuesta..." en lugar de una pantalla en blanco durante los 30 segundos que tarda la tarea. El **Human-in-the-Loop nativo** se implementa con `interrupt_before` e `interrupt_after`: el grafo se pausa antes o después de nodos específicos, persiste el estado, y espera que el operador humano apruebe la acción con `graph.invoke(Command(resume=value), config)` o la rechace. Esta funcionalidad es la base de los patrones de revisión humana que el Capítulo 08 examinará en el contexto de seguridad agéntica.

Componentes principales

- **StateGraph y AgentState:** el grafo se define tipando el estado con `TypedDict` (campos con funciones de reducción configurables) y registrando nodos con funciones Python que reciben y devuelven el estado
- **Aristas condicionales:** `add_conditional_edges(node, condition_fn, {outcome: next_node})` permite routing dinámico basado en el estado actual; la `condition_fn` es una función Python auditable y testeable independientemente
- **Checkpointing:** `MemorySaver` (en memoria, para tests), `SqliteSaver` (desarrollo local), `PostgresSaver` (producción con alta disponibilidad); persiste el estado después de cada nodo; habilita reanudación de sesiones interrumpidas
- **Streaming de pasos:** `graph.stream()` emite el estado después de cada nodo; permite mostrar el progreso del agente en tiempo real en el UI; fundamental para tareas que tardan más de 10 segundos
- **Human-in-the-loop nativo:** `interrupt_before=["node_name"]` y `interrupt_after=["node_name"]` pausan el grafo para revisión humana; reanudado con `Command(resume=value)`; el estado persiste en el checkpointer durante la pausa

Nota del Arquitecto: La curva de aprendizaje de LangGraph es real: el modelo mental de grafos de estado es diferente al de código secuencial o al de pipelines de funciones. La inversión se justifica cuando el flujo del agente tiene ciclos, cuando necesitas human-in-the-loop con persistencia de estado, o cuando la trazabilidad de cada transición es un requisito de auditoría. Si el flujo es lineal y sin ciclos, considera si realmente necesitas LangGraph o si una cadena de funciones Python con checkpointing manual sería más simple y suficiente.

Principio rector

LangGraph convierte el flujo de control del agente de un comportamiento emergente implícito a un grafo explícito e inspeccionable: cada decisión de routing es una función Python auditable, y cada estado intermedio puede ser persistido, reproducido y analizado. Esta explicitación del flujo de control es su diferenciador fundamental respecto a frameworks con abstracciones de mayor nivel.

La sección siguiente examina AutoGen, el framework de Microsoft Research que adopta un enfoque radicalmente diferente: en lugar de grafos de estado, modela la coordinación agéntica como conversaciones entre múltiples agentes con roles especializados.

Módulo 7 – Capítulo 05 – Sección 02

AutoGen: conversaciones multi-agente y colaboración entre especialistas

Mientras que LangGraph modela el comportamiento agéntico como transiciones de estado en un grafo, AutoGen lo modela como conversaciones entre agentes con roles especializados. Esta diferencia de metáfora produce frameworks con fortalezas completamente distintas: LangGraph exela en flujos de control complejos y deterministas; AutoGen exela en flujos donde la naturaleza de la colaboración entre agentes es conversacional y donde la generación de código iterativa —escribir código, ejecutarlo, analizar el resultado, corregir— es el patrón central de la tarea.

AutoGen, desarrollado por Microsoft Research, formaliza la primitiva del `ConversableAgent` : un agente que puede iniciar conversaciones, responder a otros agentes, ejecutar código en un subprocess local o Docker, y terminar la conversación cuando detecta una condición de finalización. Cada `ConversableAgent` se configura con cuatro parámetros principales: `name` (identificador del agente), `system_message` (el prompt de rol y capacidades en lenguaje natural), `llm_config` (modelo de LLM y herramientas disponibles), y `human_input_mode` que controla cuándo el agente solicita intervención humana (`NEVER` para ejecución completamente autónoma, `TERMINATE` para solicitar input cuando el agente decide que la conversación debe terminar, `ALWAYS` para solicitar confirmación en cada paso). Esta configuración declarativa hace que definir un nuevo agente especializado sea una operación de minutos.

El patrón más efectivo y documentado de AutoGen es el duo `AssistantAgent` + `UserProxyAgent` . El `AssistantAgent` es el LLM con razonamiento y herramientas: analiza el problema, genera código Python, propone análisis o explica resultados. El `UserProxyAgent` es el ejecutor: extrae bloques de código de los mensajes del `AssistantAgent` , los ejecuta en un subprocess local o en un contenedor Docker, y devuelve el resultado (stdout, stderr, código de retorno) como el siguiente mensaje en la conversación. Este ciclo —generar código, ejecutar, observar resultado, corregir si es necesario— es fundamentalmente el mismo patrón ReAct del

Capítulo 02, pero operacionalizado como una conversación bidireccional entre dos agentes con roles fijos.

La fortaleza de este patrón es su efectividad en tareas de análisis de datos y automatización de scripts: el agente puede escribir código para limpiar un dataset, ejecutarlo, ver que hay un error de tipo en la columna X, corregir el código, ejecutar nuevamente, y continuar hasta producir el análisis solicitado. Todo este ciclo ocurre sin intervención humana y con cada iteración basada en el output real de la ejecución anterior, no en razonamiento hipotético sobre qué podría pasar. El `code_execution_config` del `UserProxyAgent` permite configurar si la ejecución ocurre en un subprocess local (más simple, menos aislado) o en un contenedor Docker (más seguro, requiere Docker en el entorno).

Para orquestar más de dos agentes, AutoGen provee `GroupChat` y `GroupChatManager`. El `GroupChat` define el conjunto de participantes y las reglas de la conversación (qué agentes pueden hablar, en qué orden, cuántos turnos máximos). El `GroupChatManager` es un LLM especializado en moderación que, dado el contexto de la conversación hasta el momento, decide qué agente debe hablar a continuación. Este diseño es flexible pero introduce no-determinismo: la decisión de quién habla en cada turno es una inferencia del LLM moderador, lo que hace que el flujo de la conversación sea menos predecible que en LangGraph. AutoGen 0.4 introdujo un modelo de actores asíncrono que mejora la escalabilidad de sistemas con muchos agentes concurrentes al eliminar el bloqueo del `GroupChatManager`.

Las **conversaciones anidadas** son una característica de AutoGen que permite composición recursiva: un agente puede iniciar una sub-conversación con otro agente o grupo de agentes para resolver un subproblema, recopilar el resultado, e incorporarlo a la conversación principal. Un agente de investigación puede delegar la búsqueda de papers a una sub-conversación entre un agente de búsqueda y un agente de análisis, esperar el resultado consolidado, y continuar con la síntesis sin necesidad de gestionar los detalles de la búsqueda en la conversación principal.

Aspectos técnicos

- **ConversableAgent API:** configurado con `name`, `system_message` (prompt de rol), `llm_config` (modelo y tools), y `human_input_mode` (NEVER/TERMINATE/ALWAYS para control de intervención humana)

- **Code execution sandbox:** el `UserProxyAgent` con `code_execution_config` ejecuta código Python del `AssistantAgent` en subprocess local o en Docker; el resultado (stdout/stderr) se devuelve como el siguiente mensaje de la conversación
- **GroupChat y GroupChatManager:** para orquestar más de dos agentes; el `GroupChatManager` (LLM moderador) decide qué agente habla en cada turno basándose en el contexto de la conversación; introduce no-determinismo en el flujo
- **Termination conditions:** funciones que inspeccionan el último mensaje para decidir si la conversación debe terminar; el patrón más común es detectar la palabra "TERMINATE" o verificar que `max_turns` fue alcanzado
- **Nested conversations:** un agente puede iniciar sub-conversaciones para resolver subproblemas y reincorporar los resultados a la conversación principal; habilita composición recursiva sin contaminar el contexto de la conversación principal

***Nota del Arquitecto:** AutoGen es difícil de debuggear cuando la conversación toma caminos inesperados, especialmente en sistemas con más de dos agentes. El GroupChatManager toma decisiones de moderación que son difíciles de anticipar y de reproducir. Si la predecibilidad del flujo es un requisito, LangGraph es más apropiado. AutoGen brilla cuando el problema tiene una naturaleza conversacional intrínseca y cuando la flexibilidad del flujo es más valiosa que su determinismo.*

Para recordar

AutoGen es especialmente efectivo para flujos donde la generación de código y su ejecución deben alternarse múltiples veces: el patrón AssistantAgent-UserProxyAgent externaliza la ejecución de código del LLM a un entorno real, produciendo feedback basado en resultados reales en lugar de razonamiento hipotético, lo que reduce las alucinaciones sobre el comportamiento del código y acelera la convergencia a una solución correcta.

La sección siguiente examina CrewAI, el framework que prioriza la velocidad de desarrollo y la simplicidad de configuración sobre el control de bajo nivel, con una metáfora organizacional que hace intuitiva la definición de sistemas multi-agente para equipos con menos experiencia en frameworks agénticos.

Módulo 7 – Capítulo 05 – Sección 03

CrewAI: orquestación de equipos de agentes con roles definidos

CrewAI representa el enfoque de mayor nivel de abstracción en la ingeniería de agentes: en lugar de programar grafos de estado (LangGraph) o configurar conversaciones entre agentes (AutoGen), el desarrollador declara un equipo con roles y tareas en lenguaje natural, y el framework gestiona la coordinación. Esta filosofía de diseño prioriza explícitamente la velocidad de desarrollo sobre el control de implementación: un equipo de agentes funcional puede construirse en pocas líneas de código, a costa de menor control sobre los detalles de la coordinación entre agentes.

La metáfora organizacional de CrewAI es directa: un **Crew** es un equipo de **Agents** con roles específicos que colaboran en **Tasks** para lograr un objetivo común. Un agente de investigación puede tener el rol de "Technical Research Specialist" con el objetivo de encontrar información técnica precisa y actualizada, acceso a herramientas de búsqueda web y lectura de documentos, y una **backstory** que contextualiza su expertise ("Especialista en investigación técnica con 10 años de experiencia en análisis de documentación de APIs y papers de ingeniería"). Una tarea puede ser "Investigar las características principales de LangGraph 0.2 y producir un resumen técnico de 500 palabras con ejemplos de código". La separación entre qué hace el agente (rol) y qué debe producir en esta instancia (tarea) permite reutilizar agentes en diferentes contextos con diferentes tareas.

Los dos modos de proceso de CrewAI capturan trade-offs fundamentales en la coordinación multi-agente. **Process.sequential** ejecuta las tareas en orden, con el output de cada tarea disponible como contexto para la siguiente: el investigador completa su tarea, el analista recibe ese output y produce el análisis, el redactor recibe el análisis y produce el informe final. La secuencia es predecible y fácil de depurar; el bottleneck es que la latencia total es la suma de las latencias individuales. **Process.hierarchical** introduce un agente manager que orquesta dinámicamente la distribución de tareas: el manager analiza el objetivo general, decide qué agente debe ejecutar qué subtarea, evalúa los resultados intermedios, y puede reasignar o ajustar tareas según los resultados. Este modo añade flexibilidad adaptativa pero

el manager se convierte en un punto de fallo central cuya calidad de razonamiento determina el desempeño de todo el sistema.

La integración de memoria en CrewAI sigue un modelo de cuatro capas accesibles con la configuración `memory=True` en el `Crew`: **short-term memory** para el contexto de la conversación actual (implementado como buffer de mensajes), **long-term memory** para persistencia entre ejecuciones del crew (SQLite por defecto, configurable), **entity memory** para mantener información estructurada sobre entidades mencionadas a lo largo del trabajo (personas, empresas, conceptos clave), y **user memory** para preferencias y contexto específico del usuario. La activación de todos los tipos de memoria con una sola línea de configuración es un ejemplo de la filosofía de facilidad de uso de CrewAI, aunque con el trade-off de menor control sobre qué se almacena y cómo se recupera comparado con un sistema de memoria custom en LangGraph.

El **kickoff asíncrono** con `crew.kickoff_async()` permite ejecutar el crew sin bloquear el hilo principal, lo que es necesario en aplicaciones web que deben responder al usuario mientras el crew trabaja en background. `kickoff_for_each_async()` procesa múltiples inputs en paralelo sobre el mismo crew, lo que es especialmente útil para tareas de procesamiento batch: generar 50 reportes de análisis de competidores, procesar 100 artículos de noticias, o clasificar 200 tickets de soporte técnico de forma simultánea.

La integración nativa con el ecosistema de LangChain tools permite a CrewAI reutilizar el amplio catálogo de herramientas de LangChain sin reimplementación: las herramientas de búsqueda web, vectorstores, APIs, y utilidades de texto de LangChain se pueden pasar directamente como `tools=[]` en la definición de cada agente de CrewAI. Esta interoperabilidad reduce significativamente el tiempo de implementación cuando el proyecto ya usa herramientas de LangChain.

Componentes principales

- **Agent definition:** cada agente se define con `role` (función en el equipo), `goal` (objetivo específico), `backstory` (contexto de expertise en lenguaje natural) y `tools` (lista de herramientas disponibles); la backstory influye en cómo el LLM interpreta su rol
- **Task definition:** cada tarea específica `description` (qué debe producir), `expected_output` (formato y contenido esperado), `agent` (agente responsable), y opcionalmente `context` (resultados de otras tasks que deben considerarse como input)

- **Process.sequential vs hierarchical:** secuencial es predecible y trazable (latencia = suma de latencias individuales); jerárquico es flexible y adaptativo pero el manager LLM introduce no-determinismo en la asignación de tareas
- **Memory system:** short-term, long-term (SQLite), entity memory y user memory; activables con `memory=True` ; más simple de activar que un sistema custom, con menos control sobre la implementación
- **Kickoff asíncrono:** `crew.kickoff_async()` para ejecución no bloqueante; `kickoff_for_each_async()` para procesamiento batch paralelo de múltiples inputs sobre el mismo crew

Principio rector

CrewAI intercambia flexibilidad de bajo nivel por velocidad de desarrollo: su API declarativa permite construir un sistema multi-agente funcional en pocas líneas de código, a costa de menor control sobre la lógica de coordinación que LangGraph o AutoGen ofrecen. Es la elección correcta cuando la velocidad de prototipar importa más que el control de implementación, y cuando los flujos son suficientemente estándar para que las abstracciones de alto nivel de CrewAI sean una ventaja en lugar de una restricción.

La sección siguiente examina Pydantic AI, el framework que aborda un problema diferente: no la coordinación entre agentes ni el control del flujo, sino la garantía de tipos en las entradas y salidas del agente como mecanismo de confiabilidad en producción.

Módulo 7 – Capítulo 05 – Sección 04

Pydantic AI: agentes tipados con validación de entrada/salida

Los tres frameworks anteriores —LangGraph, AutoGen, CrewAI— abordan el problema de la coordinación y el flujo de control. Pydantic AI aborda un problema diferente y complementario: la confiabilidad de los tipos en la interfaz entre el agente y el código que lo consume. En sistemas de producción donde el output del agente alimenta directamente otros componentes del sistema —una API que espera un JSON específico, un pipeline de datos que procesa la respuesta, un formulario que se pre-llena con los resultados—, la garantía de que el output tiene el formato correcto es tan crítica como la garantía de que el contenido es correcto. Pydantic AI hace de esta garantía de tipos el principio de diseño central, no una característica adicional.

Pydantic AI es un framework de agentes desarrollado por el equipo de Pydantic que aplica la filosofía de tipado estático de Python —ya establecida en el ecosistema con modelos Pydantic para validación de datos— al dominio de los agentes. La primitiva central es el `Agent[ResultType]` donde `ResultType` es un modelo Pydantic con campos tipados: el LLM es instruido para generar output JSON que valide contra ese modelo, y si la respuesta no es válida, Pydantic AI construye automáticamente un mensaje de error descriptivo y reintenta la generación hasta `max_retries` veces antes de lanzar una excepción. Esta garantía de estructura hace que la salida del agente sea directamente deserializable a un objeto Python tipado sin transformación manual adicional.

Considerar el caso concreto de un agente de extracción de información de CVs. Con function calling directo, el desarrollador recibe una string JSON que debe parsear, validar y manejar con try/except para errores de formato. Con Pydantic AI, define:

```

from pydantic import BaseModel
from pydantic_ai import Agent

class CandidateProfile(BaseModel):
    name: str
    years_experience: int
    skills: list[str]
    education_level: str
    current_role: str | None = None

agent = Agent(
    model="anthropic:claude-3-5-sonnet-20241022",
    result_type=CandidateProfile,
    system_prompt="Extract candidate information from the provided CV text."
)

result = await agent.run(cv_text)
profile = result.data # tipo garantizado: CandidateProfile

```

El resultado `result.data` es una instancia `CandidateProfile` con validación automática de tipos: `years_experience` es garantizadamente un entero (no "five years" como string), `skills` es garantizadamente una lista (no un string separado por comas), y `education_level` está dentro de los valores permitidos si se define como `Literal["bachelor", "master", "phd"]`. Si el LLM genera output que no cumple el schema, Pydantic AI reintenta con un mensaje de error que incluye exactamente qué campos fallaron y por qué: "The field 'years_experience' must be an integer, but received '10+'. Please provide only the numeric value."

Las **herramientas tipadas** en Pydantic AI eliminan la necesidad de escribir JSON Schema manualmente. Las herramientas se definen como funciones Python con anotaciones de tipo estándar; Pydantic AI genera automáticamente el JSON Schema correspondiente y lo incluye en el contexto del LLM. Una herramienta de búsqueda se define como `async def search(query: str, max_results: int = 5) -> list[SearchResult]` y el framework genera el schema `{"type": "object", "properties": {"query": {"type": "string"}, "max_results": {"type": "integer", "default": 5}}, "required": ["query"]}`

automáticamente. Esto elimina una fuente frecuente de bugs donde el schema documentado y la implementación de la función divergen gradualmente.

El **RunContext con dependency injection** es el mecanismo por el cual las herramientas y el agente acceden a servicios externos de forma type-safe. En lugar de variables globales o closures que capturan dependencias externas, el `RunContext[Dependencies]` inyecta un objeto de dependencias tipado en cada tool call: `ctx.deps.db_connection`, `ctx.deps.http_client`, `ctx.deps.config`. Este patrón hace que las herramientas sean directamente testeables pasando dependencias mock: `agent.run(input, deps=MockDependencies(db=FakeDatabase()))`.

Pydantic AI soporta los modelos principales de la industria con una interfaz unificada: `"openai:gpt-4o"`, `"anthropic:claude-3-5-sonnet-20241022"`, `"google-gla:gemin-1.5-pro"`, `"groq:llama-3.1-70b-versatile"`, y modelos locales vía `"ollama:llama3"`. El **streaming tipado** con `agent.run_stream()` devuelve eventos intermedios como objetos tipados, no texto libre: `TextPartEvent` para texto generado, `ToolCallEvent` para invocaciones de herramientas, `ToolReturnEvent` para resultados, lo que facilita construir UIs reactivas que muestran el progreso del agente con información estructurada.

Aspectos técnicos

- **Output model typing:** `Agent[ResultType]` donde `ResultType` es un modelo Pydantic; el LLM genera output JSON que se valida y deserializa automáticamente; reintentos automáticos con mensajes de error descriptivos ante fallos de validación
- **Tool typing automático:** las herramientas se definen como funciones Python con anotaciones de tipo estándar; el JSON Schema se genera automáticamente de las anotaciones, eliminando divergencia entre schema y implementación
- **RunContext y dependency injection:** `RunContext[Dependencies]` provee acceso type-safe a servicios externos (DB, HTTP client, config); hace las herramientas directamente testeables con dependencias mock
- **Structured validation retries:** cuando el LLM genera output inválido, Pydantic AI construye un mensaje de error que especifica exactamente qué campos fallaron y por qué, y reintenta hasta `max_retries` veces antes de lanzar excepción
- **Multi-provider con interfaz unificada:** soporta OpenAI, Anthropic, Google, Groq y modelos locales (Ollama) con la misma API; cambiar de proveedor requiere modificar solo el parámetro `model`, no el código del agente

Principio rector

Pydantic AI aplica el principio de "fail fast" a la integración agéntica: en lugar de descubrir en producción que la salida del agente no tiene el formato esperado cuando el código consumidor lanza un `KeyError`, la validación en tiempo de ejecución garantiza que el output es correcto antes de que llegue al código consumidor. El costo de los reintentos de validación es menor que el costo de los bugs de integración en producción.

La sección siguiente examina los criterios para elegir entre los cuatro frameworks — LangGraph, AutoGen, CrewAI, Pydantic AI— y cuándo la opción correcta es no usar ningún framework.

Módulo 7 – Capítulo 05 – Sección 05

Criterios de selección de framework: complejidad, control y observabilidad

Haber estudiado cuatro frameworks —LangGraph, AutoGen, CrewAI, Pydantic AI— genera una pregunta práctica que ningún tutorial responde bien: ¿cómo se elige el correcto para un proyecto específico? La respuesta no es una clasificación de popularidad ni una comparación de GitHub stars. Es un análisis de tres dimensiones que el proyecto requiere concretamente: la complejidad del flujo de control, el nivel de control y determinismo necesario, y las capacidades de observabilidad disponibles. Un framework seleccionado sin este análisis produce dos tipos de problemas igualmente costosos: elegir uno demasiado simple para el caso de uso (se choca con las limitaciones del framework cuando el agente crece en complejidad), o elegir uno demasiado poderoso (overhead de abstracción innecesario que ralentiza el desarrollo y complica el debugging para un caso de uso que no lo justifica).

LangGraph es la elección correcta cuando el flujo de control del agente requiere ciclos, paralelismo, o subgrafos, o cuando la trazabilidad de cada transición de estado es un requisito de auditoría o compliance. Un agente de aprobación de créditos que puede regresar a pasos anteriores para solicitar documentación adicional, que ejecuta múltiples verificaciones en paralelo, y que debe mantener un log auditable de cada decisión tomada es un caso perfecto para LangGraph. La curva de aprendizaje es alta —el modelo mental de grafos de estado es diferente al de código secuencial— y el overhead de configuración para casos simples puede ser excesivo. No elegir LangGraph cuando el flujo es lineal y sin ciclos; para esos casos, AutoGen, CrewAI o directamente la API del proveedor son más apropiados.

AutoGen es la elección correcta cuando el patrón central del agente es generación-ejecución iterativa de código Python, o cuando se necesita orquestar múltiples LLMs en conversación donde la flexibilidad del diálogo entre agentes es más importante que el control del flujo. Un agente de análisis exploratorio de datos donde el investigador escribe código de análisis, el ejecutor lo ejecuta, y el analista interpreta los resultados en un flujo conversacional libre es el caso de uso donde AutoGen es naturalmente superior. No elegir AutoGen cuando se necesita

determinismo estricto en el flujo o cuando el debugging sistemático de la secuencia de acciones es crítico.

CrewAI es la elección correcta cuando la velocidad de desarrollo es la prioridad y el equipo no tiene experiencia profunda en frameworks agénticos, el flujo es secuencial o jerárquico con roles bien definidos, y los casos de uso son suficientemente estándar para que las abstracciones de alto nivel de CrewAI sean una ventaja. Para un MVP de un sistema de generación de contenido donde un agente investiga, uno redacta, uno revisa y uno publica, CrewAI permite tener el sistema funcionando en horas en lugar de días. No elegir CrewAI para sistemas de producción que requieren control fino sobre la lógica de coordinación o cuando los casos de borde del flujo son complejos.

Pydantic AI es la elección correcta cuando la validación de tipos de entrada/salida es crítica para la integridad del sistema, el agente se integra directamente con código Python que consume su output como objetos tipados, y el equipo tiene experiencia en tipado estático con Python. Para agentes que alimentan APIs, pipelines de procesamiento de datos, o sistemas donde un output mal formado produce fallos difíciles de diagnosticar, Pydantic AI provee garantías que ningún otro framework da. No elegir Pydantic AI como alternativa a LangGraph para flujos de control complejos; son frameworks con fortalezas complementarias, no alternativas directas.

La **opción sin framework** —usar function calling directo de la API del proveedor con código Python— es la más frecuentemente olvidada y una de las más correctas para un espacio relevante de casos de uso. Para agentes con 1-3 herramientas, flujo lineal sin ciclos, y sin requisitos de checkpointing o multi-agente, el código resultante usando directamente la API de OpenAI o Anthropic es más simple, más rápido de depurar, más fácil de mantener y más predecible que cualquier framework. El costo de adoptar un framework es real: curva de aprendizaje, debugging a través de capas de abstracción, actualizaciones de versión con breaking changes. Este costo solo se justifica cuando el caso de uso supera las capacidades del enfoque más simple.

Conceptos clave

- **LangGraph:** flujo complejo con ciclos, paralelismo, subgrafos; auditoría de cada transición; human-in-the-loop con checkpointing; curva de aprendizaje alta; no usar para flujos lineales simples

- **AutoGen**: generación-ejecución iterativa de código; orquestación conversacional multi-LLM; no usar cuando se requiere determinismo estricto del flujo o debugging sistemático de la secuencia de acciones
- **CrewAI**: velocidad de desarrollo como prioridad; flujos secuenciales o jerárquicos con roles estándar; no usar para sistemas de producción con lógica de coordinación compleja o casos de borde no estándar
- **Pydantic AI**: validación de tipos de output como garantía de confiabilidad; integración con código Python tipado que consume directamente el output del agente; complementa LangGraph, no lo reemplaza
- **Sin framework (API directo)**: la opción más simple y frecuentemente la más correcta para agentes con 1-3 herramientas y flujo lineal; menor overhead, código más legible, menor costo de mantenimiento

Nota del Arquitecto: La presión del equipo para adoptar el framework más popular o el más sofisticado es real pero contraproducente. He visto proyectos que implementaron LangGraph para un agente de 2 herramientas con flujo lineal, produjeron código difícil de leer para los stakeholders, y tuvieron bugs de versión que tardaron días en diagnosticar. La misma funcionalidad implementada directamente con la API de Anthropic habría sido 5 veces menos código y depurable en minutos. El mejor framework es el mínimo suficiente para el problema, evaluado honestamente contra el caso de uso real, no contra el caso de uso más sofisticado que se podría imaginar.

Para recordar

El mejor framework de agentes para un proyecto es el mínimo necesario para resolver el problema: empezar con la API directa del proveedor y agregar el framework solo cuando el caso de uso supera las capacidades del approach más simple. Esta regla previene el sobrediseño y mantiene el código legible para el próximo desarrollador que lo va a leer.

La sección siguiente cierra el capítulo de frameworks con la distinción fundamental que integra los cuatro frameworks estudiados en una perspectiva unificada: el framework organiza los componentes del agente, pero no determina su inteligencia.

Módulo 7 – Capítulo 05 – Sección 06

Cierre: el framework no define la inteligencia del agente — solo su estructura

El capítulo sobre frameworks de agentes deja una conclusión que conviene articular con precisión: LangGraph, AutoGen, CrewAI y Pydantic AI son herramientas de ingeniería de software, no de inteligencia artificial. Determinan cómo se organiza el flujo de control, cómo se comunican los componentes, y qué garantías de tipo y persistencia se proveen; pero no determinan la calidad del razonamiento del agente, la precisión de sus herramientas, o la relevancia de su memoria. El framework es el andamiaje; la inteligencia está en el LLM, los prompts, las herramientas y la memoria —componentes que funcionan bien o mal independientemente del framework que los organiza.

Esta distinción tiene consecuencias prácticas directas. Un agente construido con LangGraph pero con prompts deficientes, herramientas mal descritas, y sin gestión de memoria producirá peores resultados que un agente simple construido con function calling directo pero con prompts cuidadosamente diseñados, herramientas bien documentadas y un sistema de memoria apropiado para el caso de uso. La tentación de esperar que el framework resuelva problemas de calidad que son inherentemente problemas de diseño de componentes es una de las causas más frecuentes de proyectos agénticos que no llegan a producción.

La elección del framework, por tanto, debe ser la última decisión de diseño, no la primera. El proceso correcto es: primero, definir con precisión el comportamiento requerido del agente — qué tareas debe completar, con qué criterios de éxito, con qué SLOs de latencia y costo—; luego, identificar los componentes necesarios —qué herramientas, qué tipo de memoria, qué estrategia de razonamiento—; finalmente, elegir el framework que mejor organiza esos componentes dadas las capacidades del equipo y los requisitos operativos del sistema. Este orden de decisiones produce sistemas coherentes donde el framework amplifica las capacidades del equipo en lugar de añadir complejidad sin propósito.

La transición al capítulo siguiente es natural: una vez que el agente tiene sus componentes diseñados y su framework elegido, la siguiente pregunta es cómo verificar que funciona

correctamente antes de desplegarlo en producción. El Capítulo 07 sobre testing de agentes aborda exactamente ese desafío, con las particularidades que hacen el testing agéntico fundamentalmente diferente al testing de software convencional.

Para recordar

El framework es el andamiaje; la inteligencia del agente está en el LLM, los prompts, las herramientas y la memoria —componentes que funcionan bien o mal independientemente del framework que los organiza. Elegir el framework antes de diseñar los componentes produce el orden incorrecto de decisiones.

"A fool with a tool is still a fool." — Grady Booch, pionero de la ingeniería de software orientada a objetos; en el contexto agéntico, ningún framework —por sofisticado que sea— compensa prompts ambiguos, herramientas mal descritas, o ausencia de estrategia de memoria en un sistema de agentes en producción.